

1.What We Built :

Axon is an AI-powered Industrial Knowledge Intelligence platform. It ingests heterogeneous plant documents - maintenance logs, inspection reports, SOPs, engineering drawings, scanned registers - and turns them into a single connected knowledge graph that a conversational copilot can reason over. Instead of an engineer searching folders, emails, and asking senior colleagues, they ask a question in plain language and get a grounded, source-cited answer.

Concretely, the working system does five things end to end:

Universal ingestion - accepts PDF, DOCX, XLSX, and image documents; runs native text extraction where possible and falls back to OCR for scanned pages.

Semantic search - chunks and embeds every document (Gemini text-embedding-004) into Qdrant, so a query retrieves the most relevant passages regardless of exact wording.

Knowledge graph construction - an LLM extraction pass reads each document and pulls out equipment, personnel, locations, failures, regulations, and dates, plus relationships between them (e.g. Pump P204 --HAD_FAILURE--> Bearing damage), merging everything into one graph so the same equipment mentioned across many documents becomes a single connected node.

Grounded Q&A copilot - fuses vector search results with graph relationship context before asking the LLM to answer, returning the answer with a confidence level and the specific source documents used.

Failure Pattern Engine - walks the graph for equipment with two or more dated failures, computes the interval between them, and where the interval is regular, predicts the next likely failure window with the full evidence chain attached. This is deliberately not a black-box model: every number in its output traces back to a document.

2. How Axon Maps to the Evaluation Focus:

2.1 Entity extraction accuracy across document types

`entity_extraction_service.py` sends each document's extracted text to Gemini with a structured prompt asking for six entity types (equipment, personnel, locations, failures, regulations, dates) plus a relationships list, returned as strict JSON. The prompt explicitly instructs the model not to invent entities absent from the text. This runs uniformly across every supported document type - PDFs (native and OCR'd),

Word, Excel - because extraction operates on normalized OCR output, not the file format.

How we propose to measure it: precision/recall against a manually labeled sample 15–20 representative documents, hand-marked by a domain-literate reviewer, compared against pipeline output, reported per entity type.

Current status, honestly: validated qualitatively on representative sample documents during development; reliably picks up clearly-stated entities and relationships. We have not run the formal precision/recall benchmark against real plant documents, since we don't have proprietary access to any. If real (even redacted) documents become available before submission, running that benchmark is roughly an afternoon of work and would let this section quote real numbers instead of a proposed protocol.

2.2 Query answer quality on domain-expert benchmark questions

What's built: /chat/ask combines top-k vector search hits with a one-hop graph neighbourhood around any matched entities, and asks the LLM to answer using only that context — explicitly instructed to decline rather than guess when context is insufficient. Returns a confidence label (from top similarity score) and the exact source filenames used.

Proposed benchmark protocol: a ten-question starter set (Appendix A) spanning factual lookup, causal/RCA-style reasoning, and cross-document synthesis. Score each answer: correct and grounded / partially correct / incorrect or hallucinated / correctly declined.

Current status: exercised manually with representative questions; produces grounded, cited answers when the corpus contains the answer. A scored run against the ten-question set on real documents is the natural next step (under an hour once documents are available).

2.3 Knowledge graph linkage completeness

What's built: the graph store merges entities by exact name match across documents, so the same equipment mentioned in three files becomes one node with three sources, not three disconnected nodes. stats() reports node/edge counts and a type breakdown.

How we'd measure completeness: (a) coverage — % of uploaded documents contributing at least one entity to the graph; (b) connectivity — average relationships per equipment node, which should rise as more documents reference the same equipment.

Known limitation, stated plainly: linkage relies on exact string matching today. "Pump P204," "Pump-204," and "P204" would currently create three separate nodes instead of one. Fuzzy entity resolution is the highest-value next engineering step and is flagged in the roadmap rather than glossed over.

2.4 Time-to-answer versus traditional search

What's built: a single /chat/ask call (embed → vector search → one graph hop → generate) completes in a small number of API round trips — a matter of seconds, regardless of corpus size, since Qdrant's search cost doesn't scale linearly the way manual search does.

The comparison point: the challenge brief itself quantifies the baseline — professionals in asset-intensive industries lose a large share of working hours to search that already exists somewhere in the organisation, with the brief's own example (folders → emails → maintenance software → manuals → colleagues → another plant) taking hours to days, sometimes never resolving.

How we'd validate this rigorously: a small controlled study — the same question, half the group using traditional search, half using Axon, timed to a correct answer. We haven't run this study; the comparison above is architectural reasoning, not a measured result, and we're flagging that distinction rather than presenting an assumption as data.

2.5 Compliance gap detection accuracy

Honest status: not implemented. This is the one evaluation dimension we made a deliberate scope cut on. A compliance agent that reliably flags real gaps needs a labeled regulatory corpus and careful false-positive control — getting it wrong is worse than not having it, since a missed or false compliance flag in a safety context has real consequences.

Why this wasn't a wasted design decision: the graph already models Regulation as a first-class node type with relationships like Procedure --REFERENCES-->

Regulation, specifically so a compliance agent is additive on top of the existing graph, not a redesign. Building it: (1) ingest regulatory documents the same way any document is ingested, (2) walk the graph for equipment/procedures missing or referencing outdated regulations, (3) surface gaps with the same evidence-trail pattern the Failure Pattern Engine already uses.

How accuracy would be measured, once built: precision/recall against a test set with deliberately-planted compliance gaps — same evaluation shape as entity extraction.

2.6 Demonstrated improvement in cross-functional knowledge discovery

What's built: one graph and one vector index across every document type — rather than siloed tools per department — means a single query can synthesize across maintenance, safety, and personnel data. E.g. "which pumps has Ravi Kumar inspected, and did any fail afterward?" requires joining an inspection record against a maintenance record, two document types that typically live in separate systems.

How we'd demonstrate this concretely: upload a small set spanning at least three document types for the same equipment, then run a query that only makes sense combining at least two. The graph view lets a judge visually confirm the linkage — one equipment node with edges into both a maintenance and an inspection document.

Current status: exercised qualitatively with representative sample data structured like real documents, not proprietary plant data (we don't have access to any). Stated explicitly rather than implying the demo used real industrial documents.

Innovation : Axon builds an actual knowledge graph from unstructured documents rather than treating them as an undifferentiated pile of RAG chunks, and its predictive-maintenance feature is explainable by construction - every prediction is a direct read of dated failure events already in the graph, not an opaque model score.

Business Impact: Directly targets the costs the problem briefly quantifies: hours lost to search, downtime linked to fragmented equipment history, and knowledge loss as experienced engineers retire - without requiring new sensors or instrumentation.

Technical Excellence: A real, running pipeline: upload → OCR → chunking → embedding → entity extraction → graph construction → grounded Q&A, all implemented, every backend file under ~170 lines. The networkx-over-Neo4j choice was made as deliberate engineering judgement — removing infrastructure risk from a live demo with the graph service written so a Neo4j migration later is a same-shape swap, not a rewrite.

Scalability: Qdrant already supports sharding; the graph service's function signatures don't change on a Neo4j swap; document processing is stateless and queue-ready; file_id/sources metadata is already the seam needed for multi-plant tenant scoping.

User Experience: One upload triggers the full pipeline automatically; chat, graph, and dashboard are each one click away; every answer surfaces its own confidence and sources inline.

3.Scope: What's Implemented vs Roadmap

FEATURE	STATUS
Universal document ingestion (PDF/DOCX/XLSX/images, OCR fallback)	Implemented
Semantic search (chunking, embeddings, Qdrant)	Implemented
Knowledge Graph Agent (entity + relationship extraction)	Implemented
Industrial AI Copilot (RAG + graph-grounded chat, confidence + sources)	Implemented
Failure Pattern Engine (explainable predictive maintenance)	Implemented
Executive Dashboard (graph stats, top predictions)	Implemented
Fuzzy entity resolution (equipment tag aliasing)	Roadmap
Compliance & Regulatory Gap Detection Agent	Roadmap - additive on existing graph, see 2.5
Automated Report Generator (PDF/audit export)	Roadmap
Authentication / multi-tenant access control	Roadmap
Native mobile app	Roadmap (current UI is responsive web)

